

Predicting Late Sprint Closure and Unfinished Committed Scope from Reconstructed Jira Histories

Hani AbuSharkh

Higher Colleges of Technology
United Arab Emirates
habusharkh@hct.ac.ae

Maryam Hanif

United Arab Emirates University
United Arab Emirates
700052696@uaeu.ac.ae

Rafat Damseh

United Arab Emirates University
United Arab Emirates
rdamseh@uaeu.ac.ae

Abstract—Repository data are useful at Sprint Planning only if the features reflect what was known at that time. We examine two repository-derived outcomes in eight TAWOS projects: late sprint closure and unfinished scope that was committed when the sprint began. Sprint membership and story points are reconstructed from Jira change history, with a documented static fallback when no relevant history is recorded. After removing nonclosed and invalid sprint rows and one genuine cold start per project, 2,602 sprints remain. Late closure is only moderately discriminable: random forest reaches 0.705 ROC-AUC on the main time-ordered split, with a project-cluster 95% interval of 0.604–0.783. For nonempty sprints, a simple scope-only logistic model reaches 0.885 ROC-AUC, compared with 0.852 for random forest. The simpler model therefore captures most of the useful spillover ranking signal. Provenance matters: when the analysis is restricted to the much smaller subset in which every committed item has observed Sprint and Story Points history, the random-forest AUC falls to 0.652 on 110 test sprints. Results are also reported across multiple temporal cut points and held-out projects. The evidence supports cautious retrospective ranking of these two outcomes, not a deployment-ready Scrum decision system.

Index Terms—Scrum, sprint analytics, Jira, temporal provenance, software analytics, machine learning, data leakage

I. INTRODUCTION

Sprint Planning is one of the few points in Scrum where the team can still change the amount and shape of the work before execution begins. By the time blockers appear, a burndown stops moving, or unfinished work is repeatedly discussed in the Daily Scrum, the sprint is already in progress. We therefore treat sprint start as a hard cutoff for the prediction data. Anything recorded or changed after that point is not used as if it had been known during planning.

We use two specific outcomes rather than a broad measure of sprint success. A sprint is labeled *late sprint closure* when Jira records completion more than one day after the planned end. *Unfinished committed scope* means that at least one issue

present when the sprint started is still unresolved at the planned end. These labels describe repository events only; they should not be read as measures of the team’s overall performance.

A final Jira export is not a historical snapshot. Issues move between sprints, estimates change, and some records are created after a sprint has already started. To reduce that timing error, we reconstruct Sprint membership and Story Points from timestamped TAWOS change records. When a field has no relevant history, the static value is used as a fallback and is reported as such. We exclude developer count because assignee-at-start cannot be reconstructed reliably from the available export.

Four questions guide the analysis: (1) how much ranking signal is present for late closure and unfinished committed scope; (2) whether a simple scope model performs as well as more flexible models; (3) how stable the findings are across projects and temporal cut points; and (4) how sensitive the results are to incomplete historical provenance and common leakage mistakes.

II. RELATED WORK

Machine learning in Agile software analytics has most often been used for effort estimation, sprint monitoring, and project-risk support. Choetkiertikul et al. [2] estimated story points from Jira text, while Shankar et al. [3] compared several predictive models for Agile effort data. Pérez Castillo et al. [4] classified sprint performance using burndown information, which is observed after the sprint has begun. Obike et al. [5] focused on sprint deliverability with an ensemble method, while Almalki [6] examined interpretable AI for Agile project-risk support.

The real question is narrower: did we actually have access to these predictors at the time the sprint began? To check this, we reconstruct time-varying fields as they would have looked at that point, run simple baselines alongside random forest and XGBoost, and evaluate on both later sprints and entirely held-out projects. This matters because it stops the models from getting credit for final-state values they couldn’t have known during planning.

Table 1 compares the studies we selected against our own

TABLE I: Selected related work and the evaluation gap

Study	Main task	Sprint-start features	Project holdout
Choetkiertikul et al.	Story-point estimation	Yes	No
Shankar et al.	Effort/time estimation	Yes	No
Pérez Castillo et al.	Sprint performance	No; burn-down	No
Obike et al.	Deliverability	Intended	No
Almalki	Agile risk support	Historical	No
This work	Closure / unfinished scope	Reconstructed	Yes

TABLE II: Sprint flow by project

Project	Raw	Nonclosed	Invalid	Modeled	Nonempty
Apache Mesos	227	2	0	224	133
Appcelerator Studio	163	0	0	162	129
Confluence Cloud	477	10	2	464	197
Confluence Server	565	7	0	557	289
Lsstcorp Data	396	8	0	387	309
Mule	311	13	0	297	228
Titanium SDK	301	1	0	299	241
Titanium Mobile	217	3	1	212	165

setup, focusing especially on what’s being predicted, when the inputs were captured, and whether evaluation held out whole projects.

III. DATA AND RECONSTRUCTION

We use eight open-source projects from TAWOS [1], each with at least 30 sprint records. Altogether, the selected projects give us 2,657 sprint rows and 145,309 issues. Table 2 shows how those sprint rows get filtered down before modeling. FUTURE and ACTIVE rows are removed before labels are built because their closure outcome is not observed. Three CLOSED rows with impossible timestamps are also removed. Historical features require a predecessor, so the first eligible sprint in each project is the only cold start excluded. After these exclusions, 2,602 sprints remain for modeling.

TAWOS uses two different sprint identifiers: the database ID and the Jira sprint ID. We build project-level maps for both before tracing issue history. Starting from the latest recorded state, the code walks each issue’s changes backward to the sprint-start time. An issue is included in the committed set only if it already existed, belonged to that sprint at that time, and was not resolved before the start. Story Points are reconstructed in the same way. This produces 20,603 issue-sprint commitments across the eligible sprints.

Not every issue has a recorded Sprint or Story Points change. When a change is missing, we fall back to the static value and treat it as unchanged. This is an assumption; the data cannot confirm that the field truly stayed constant. Since coverage differs across projects, we report the provenance of the commitments and rerun the spillover analysis on subsets with observed history.

The reconstruction is applied in the same order for every

TABLE III: Commitment provenance by project

Project	Sprint hist.	Sprint fallback	SP hist.
Apache Mesos	96.4%	3.6%	56.8%
Appcelerator	95.8%	4.2%	87.2%
Confluence Cloud	54.1%	45.9%	11.2%
Confluence Server	99.1%	0.9%	54.4%
Lsstcorp Data	92.4%	7.6%	41.8%
Mule	92.3%	7.7%	77.0%
Titanium SDK	97.1%	2.9%	68.3%
Titanium Mobile	93.6%	6.4%	54.0%

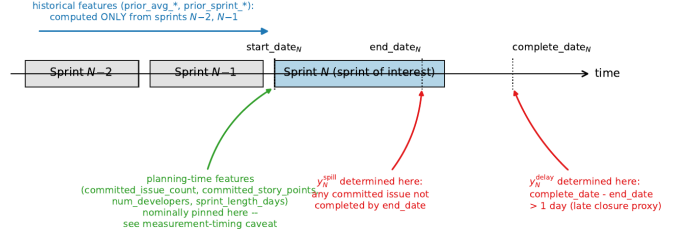


Fig. 1: Data timing and reconstruction used before model fitting.

issue. Change events are sorted by issue, timestamp, and change ID. The state begins from the latest recorded value and is rolled backward to each sprint start. A change recorded exactly at the start timestamp is treated as available at the start; only later changes are undone. Multi-sprint Jira values are parsed as sets, and unknown sprint identifiers are ignored if they do not map to a retained sprint.

1. Load Sprint, Issue, and Change_Log rows for the eight projects.
2. Remove nonclosed and invalid sprint rows before any labels are created.
3. Build project-specific maps for database sprint IDs and Jira sprint IDs.
4. Reconstruct Sprint membership and Story Points at each sprint’s start time.
5. Keep an issue only if it existed by the start time, belonged to that sprint, and had not already been resolved.
6. Aggregate committed issues, committed points, completed issues, delivered points, and carryover.
7. Build labels and historical features, then remove one genuine cold start per project.

IV. METHODS

A. Outcomes and features

Late closure is defined as

$$y_s^{\text{late}} = \mathbb{I}[c_s - e_s > 1 \text{ day}], \quad (1)$$

where c_s is Jira’s completion timestamp and e_s is the planned end timestamp. Missing completion dates are never coded as on time; the corresponding nonclosed sprints are excluded before modeling.

TABLE IV: Predictor groups used in the final analysis

Group	Variables
Current scope	committed issue count; committed story points; planned sprint length
Recent history	mean delivered points over prior 3 sprints; mean completion ratio over prior 3; prior carry-over; prior sprint completion ratio
Context	one-hot project identity in full models only

TABLE V: Model settings held fixed across the reported runs

Model	Main settings
Logistic regression	standardized numeric features; balanced class weights
Random forest	300 trees; max depth 5; min leaf 3; balanced class weights
XGBoost	300 trees; max depth 4; learning rate .05; log-loss objective

Unfinished committed scope is defined as

$$y_s^{pill} = \mathbb{1}[u_s > 0], \quad (2)$$

where u_s is the number of sprint-start committed issues unresolved by the planned end. The main conditional analysis excludes reconstructed zero-scope sprints because they are negative by construction.

Current-sprint predictors are committed issue count, committed story points, and planned sprint length. Historical predictors summarize recent delivered points, recent completion ratio, prior carryover, and the immediately preceding sprint’s completion ratio. Project identity is included only in the full models. The scope-only logistic model uses current scope and sprint length, with no project identity or historical variables.

B. Models, thresholds, and validation

We compare prevalence baselines, count-only and scope-only logistic regression, random forest, and XGBoost. Random forest uses 300 trees, maximum depth 5, and minimum leaf size 3. XGBoost uses 300 trees, maximum depth 4, learning rate 0.05, and log-loss optimization. Logistic regression and random forest use balanced class weights; XGBoost is unweighted. Precision, recall, and F1 use a fixed 0.5 threshold chosen before testing. We use ROC-AUC as our main measure of ranking performance. Brier scores and reliability bins are included as descriptive checks, since class weighting changes how the raw probabilities should be read.

For the main test, we order each project’s sprints chronologically, training on the earlier 70% and testing on the later 30%. We also repeat this at 50/50, 60/40, and 80/20 splits. In leave-one-project-out (LOPO) testing, we hold out one entire project from training and test on it separately — though its earlier sprints still feed the historical features, so this measures how well the model transfers across projects rather than a true cold start on a brand-new project.

For uncertainty on the main split, we resample projects as whole units rather than sampling individual sprint rows. This keeps sprints from the same project together. Feature

importance is measured by permuting variables on held-out data rather than using the forest’s built-in impurity scores alone.

V. RESULTS

The outcome rates are not similar across the eight projects. Late closure ranges from 19.2% in Mule to 59.9% in Titanium SDK, and conditional spillover is above 87% in every project. Because of this imbalance, a single pooled threshold score can hide project-level differences. It also means that an always-positive rule can obtain a high F1 for conditional spillover.

A. Cohort characteristics

The descriptive numbers already separate the two outcomes. Across the 2,602 modeled sprints, starting scope averages 7.9 issues and 35.0 story points. Late and on-time sprints are fairly close in scope and planned duration. Spillover is different: many negative cases have no reconstructed committed work at all, so the gap between positive and negative sprints is much larger. For that reason, we report a separate analysis that keeps only nonempty sprints.

B. Main test results

On the main 70/30 split, 246 of 786 test sprints are late. Random forest gives the highest late-closure ROC-AUC, 0.705, with an F1 of 0.562. The scope-only logistic model manages just 0.530 AUC — barely above chance. This suggests starting scope alone tells us little about whether a sprint will close late, whereas project and historical context contribute at least some useful signal. Even with that extra information, the ranking remains moderate.

For nonempty spillover, 482 of 511 test sprints are positive. Because almost every sprint is positive, an always-positive rule already gets F1=0.971; F1 is therefore not a useful headline by itself. The scope-only logistic model reaches 0.885 ROC-AUC, compared with 0.852 for the full random forest. In this test, most of the ranking information is already in the starting scope, and the larger model does not improve AUC.

C. Uncertainty and alternative time splits

Project-level resampling gives a 95% AUC interval of 0.604–0.783 for the late-closure random forest. For nonempty spillover, the interval is 0.820–0.922 for scope-only logistic regression and 0.797–0.914 for random forest. The intervals overlap, so the 0.033 AUC gap on the main split is not strong evidence that one model will always outperform the other.

The forward splits are consistent with that picture. Late-closure random-forest AUC varies from 0.669 at 50/50 to 0.711 at 80/20. For nonempty spillover, random forest stays between 0.846 and 0.856, while scope-only logistic regression ranges from 0.865 to 0.887. The result is therefore not specific to the 70/30 split.

TABLE VI: Project-level modeled cohort and outcome prevalence

Project	Modeled	Zero scope	Late positives	Late rate	Nonempty	Spillover positives / rate
Apache Mesos	224	91	66	.295	133	128 / .962
Appcelerator Studio	162	33	73	.451	129	121 / .938
Confluence Cloud	464	267	186	.401	197	187 / .949
Confluence Server	557	268	171	.307	289	255 / .882
Lsstcorp Data	387	78	226	.584	309	304 / .984
Mule	297	69	57	.192	228	200 / .877
Titanium SDK	299	58	179	.599	241	228 / .946
Titanium Mobile	212	47	48	.226	165	152 / .921

TABLE VII: Planning-time feature means over 2,602 modeled sprints

Group	Issues	Story points	Days
Overall	7.9	35.0	13.8
Late closure = 0	7.3	32.9	13.2
Late closure = 1	8.8	38.3	14.6
Spillover = 0	0.3	1.1	12.8
Spillover = 1	12.9	57.0	14.4

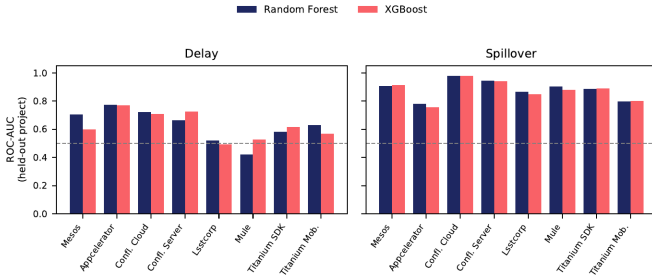


Fig. 2: Held-out-project AUC by project.

D. Results across projects and feature evidence

LOPO results vary substantially by project. Late-closure random-forest AUC ranges from 0.531 to 0.780. Conditional-spillover random forest ranges from 0.706 to 0.950. Table 11 includes denominators because project prevalence differs considerably. A model that transfers well to one project may be only modestly useful on another.

Held-out permutation importance shows what the random forest is actually relying on. For late closure, project identity (mean AUC drop of 0.104) and sprint length (0.077) matter most. For conditional spillover, committed issue count dominates by far, with a mean AUC drop of 0.220 — the next-largest drop is only 0.021. This lines up with what the scope-only baseline already suggested, which makes the result easier to trust.

A feature-group ablation tells the same story. For late closure, keeping only planning-time variables barely changes performance (0.706 AUC versus 0.705 for the full model), and dropping project identity entirely still gives 0.696. For full-population spillover, random forest stays essentially unchanged at 0.987 AUC, while using historical features alone drops performance to 0.697. In other words, historical features can't rescue the spillover prediction once you already know the initial scope.

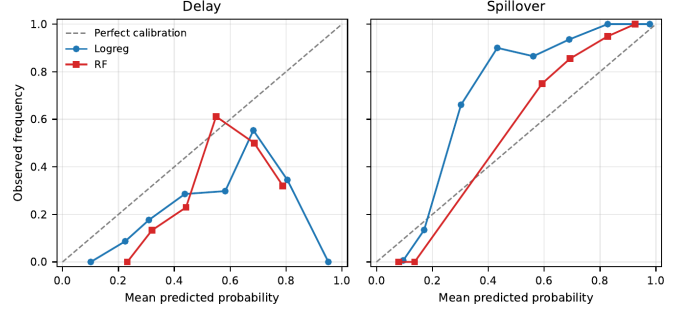


Fig. 3: Reliability bins for selected random-forest scores.

E. Provenance sensitivity

Our main reconstruction blends observed change history with a static fallback wherever no change is recorded, and that coverage varies a lot by project — 96.4% of Apache Mesos commitments have observed Sprint history, for instance, versus just 54.1% for Confluence Cloud. Story-point history is even sparser.

Table 13 narrows the conditional spillover analysis down to subsets with stronger provenance. Requiring observed Sprint history for every committed item cuts the sample to 1,174 modeled sprints (356 in the test set), where scope-only logistic regression still holds at 0.875 AUC and random forest at 0.850. Requiring both Sprint and Story Points history shrinks things further, to just 358 modeled sprints and 110 test sprints. On this stricter subset, scope-only AUC drops to 0.772 and random forest to 0.652. The smaller sample means less stable estimates, but the drop itself is worth taking seriously — it shows that provenance quality changes how hard the task actually looks.

F. Calibration and leakage checks

Reliability bins make clear that these raw probabilities shouldn't be treated as literal risk percentages. That's partly a consequence of how common conditional spillover is and partly a consequence of class weighting. We treat the probability scores mainly as a way to rank sprints, not as calibrated risk estimates.

Two controlled leakage checks show how easily evaluation choices can distort the results. Adding the current sprint's own completion ratio as a feature barely moves late-closure F1 but pushes spillover F1 nearly to 1. Oversampling before the train/test split, meanwhile, substantially inflates late-closure F1 while doing almost nothing to the spillover task, which

TABLE VIII: Main time-ordered test results. Thresholded metrics use a fixed 0.5 threshold.

Outcome	Model	n	Pos.	Prev.	F1	ROC-AUC	PR-AUC	Brier
Late closure	Always positive	786	246	.313	.477	.500	.313	.687
Late closure	Scope-only LR	786	246	.313	.336	.530	.373	.244
Late closure	Random forest	786	246	.313	.562	.705	.458	.223
Late closure	XGBoost	786	246	.313	.535	.695	.478	.206
Spillover, nonempty	Always positive	511	482	.943	.971	.500	.943	.057
Spillover, nonempty	Count-only LR	511	482	.943	.650	.881	.989	.234
Spillover, nonempty	Scope-only LR	511	482	.943	.696	.885	.992	.230
Spillover, nonempty	Random forest	511	482	.943	.875	.852	.990	.136

TABLE IX: Project-cluster 95% intervals on the main split

Outcome	Model	F1 interval	AUC interval
Late closure	RF	[.317,.688]	[.604,.783]
Spillover	Scope LR	[.482,.823]	[.820,.922]
Spillover	RF	[.804,.923]	[.797,.914]

TABLE X: Temporal sensitivity. AUC values on forward test periods.

Outcome/model	50/50	60/40	70/30	80/20
Late closure, RF	.669	.685	.705	.711
Spillover, scope LR	.865	.887	.885	.878
Spillover, RF	.846	.856	.852	.853

is already highly imbalanced toward positives. So leakage doesn't inflate results by some fixed amount — how much damage it does depends on the target and the specific mechanism involved.

G. Outcome-definition sensitivity

The broad findings are not tied to one exact threshold. For late closure, random-forest AUC is 0.696 with no grace day, 0.705 with the one-day definition, 0.685 with a three-day tolerance, and 0.626 with seven days. In the full spillover population, requiring at least two or three unfinished issues still produces AUCs of 0.984 and 0.980. A story-point completion-ratio label below 0.8 gives 0.977 AUC. These checks do not remove the provenance and prevalence concerns, but they show that the full-population separation is not created by a single arbitrary cutoff.

H. Reconstruction audit

The exported change log contains 88,364 relevant Sprint and Story Points events. At issue level, history coverage varies substantially: 38.1% of Titanium Mobile issues have Sprint history, compared with 2.9% in Confluence Cloud. Commitment-level coverage is higher because committed items are not a random sample of all project issues. Both project coverage and stricter subset results are reported because missing history is not uniform across the data.

Several edge cases are handled explicitly. Multi-valued Sprint fields are treated as sets. An issue can carry a historical sprint identifier that belongs to a sprint outside the retained subset; that identifier is ignored instead of being forced into a selected sprint. Missing story points contribute zero points

TABLE XI: LOPO random-forest AUC with test denominators

Project	Late n/pos	AUC	Spill n/pos	AUC
Apache Mesos	224/66	.572	133/128	.858
Appcelerator	162/73	.780	129/121	.950
Confluence Cloud	464/186	.745	197/187	.706
Confluence Server	557/171	.604	289/255	.768
Lsstdcorp Data	387/226	.557	309/304	.937
Mule	297/57	.561	228/200	.809
Titanium SDK	299/179	.574	241/228	.873
Titanium Mobile	212/48	.531	165/152	.780

TABLE XII: Random-forest feature-group ablation, F1 (AUC)

Configuration	Late closure	Full spillover
Full	.562 (.705)	.969 (.986)
Planning only	.588 (.706)	.969 (.987)
History only	.377 (.625)	.680 (.697)
No project identity	.542 (.696)	.969 (.989)

but do not remove an otherwise valid commitment. Issues resolved before the sprint starts are excluded from planning scope because they cannot represent unresolved work at that time.

VI. DISCUSSION

The clearest finding is not that a complex model wins. It is that the two repository outcomes behave differently. Late closure contains a modest signal that depends on project context and sprint length, and that signal transfers unevenly. Unfinished committed scope is easier to rank, but most of that ranking comes from the amount of work present at sprint start, so a transparent scope-only model earns its place as a real baseline here, not just a formality.

This also narrows what we're actually claiming. The analysis can rank historical sprints under our stated reconstruction assumptions — it does not show that a Scrum Master should act on any given score, that using the score would improve delivery, or that the model would calibrate well in a different organization. A more sensible next step would be a silent prospective deployment: generate scores without letting them influence team behavior, calibrate locally, and only then consider an actual intervention study.

The provenance sensitivity is equally important. Reconstructing timestamped fields is more defensible than using a static export, but incomplete change history remains an identification problem. The strict observed-history subset is smaller and produces weaker random-forest discrimination. Any use

TABLE XIII: Conditional-spillover sensitivity to observed history

Subset	Model	Test n	Pos.	F1	AUC
Observed Sprint	Scope LR	356	335	.719	.875
Observed Sprint	RF	356	335	.875	.850
Sprint + SP history	Scope LR	110	94	.652	.772
Sprint + SP history	RF	110	94	.876	.652

TABLE XIV: Selected random-forest reliability bins

Outcome	Score bin	n	Mean	Observed
Late	.3-.4	159	.356	.182
Late	.5-.6	144	.544	.625
Late	.7-.8	46	.749	.348
Spillover	.3-.4	49	.350	.898
Spillover	.7-.8	81	.752	.975
Spillover	.9-1.0	109	.947	1.000

of the main estimate should therefore keep the fallback assumption visible.

VII. THREATS TO VALIDITY

Construct validity. Jira’s completion timestamp marks when the sprint was closed, not necessarily when the work itself was finished. The spillover label is similarly narrow — it only captures whether at least one item committed at sprint start was still unresolved by the planned end. Neither label should be treated as a stand-in for overall team performance.

Reconstruction validity. Missing Sprint or Story Points history is handled with static fallback. The provenance tables and sensitivity analysis show how much the result depends on that choice, but they cannot recover events that were never recorded. Jira use also differs by project.

Evaluation validity. Only eight projects are available, so project-level uncertainty is necessarily wide. LOPO retains local historical features from the held-out project and should be interpreted as transfer of the learned rule rather than a complete cold start. Thresholded metrics are affected by prevalence, especially for conditional spillover.

External validity. The projects are open-source Jira projects. Enterprise teams may differ in staffing, backlog practice, and administrative use of sprints. Absolute performance should not be transported without local validation.

VIII. REPRODUCIBILITY AND AUDIT TRAIL

The artifact keeps the data preparation and modeling stages separate so that a reader can inspect the reconstruction before looking at model scores. The raw project subset consists of three CSV exports: Sprint, Issue, and the relevant Sprint/Story Points rows from Change_Log. The reconstruction code writes the planning-time aggregates used by the models, while the analysis scripts save their outputs as CSV or JSON rather than only printing them to the console.

The main result tables can be regenerated with three commands. `python -m scripts.run_baseline` rebuilds the principal temporal and held-out-project results. `python -m`

TABLE XV: Leakage demonstrations, F1 before and after the mistake

Outcome	Mechanism	Correct	Leaky	Change
Late	Post-outcome feature	.463	.473	+.010
Late	Oversample before split	.457	.646	+.189
Spillover	Post-outcome feature	.756	.991	+.235
Spillover	Oversample before split	.972	.972	.000

TABLE XVI: Selected label-sensitivity results, random forest

Variant	Test prevalence	F1	AUC
Late >0 days	.609	.649	.696
Late >1 day	.313	.562	.705
Late >3 days	.182	.353	.685
Late >7 days	.111	.216	.626
Spillover ≥ 2 issues	.467	.955	.984
SP completion $<.8$	.467	.957	.977

`scripts.audit_commitment_provenance` creates the provenance summaries and the stricter-history sensitivity sets. `python -m scripts.run_validation_sensitivity_analyses` produces the transparent baselines, project-cluster intervals, forward cut points, project-flow counts, held-out permutation importance, calibration bins, and the recorded software environment. Label sensitivity and leakage demonstrations are kept in separate scripts so that they do not alter the primary results.

Automated tests cover label construction, cold-start handling, reconstruction of planning snapshots, feature creation, scope decomposition, and known leakage cases. The package also includes a manuscript-number crosswalk that points each headline statistic to the result file from which it was taken. This is a practical safeguard against one manuscript being updated while another retains an older number.

Exact bitwise agreement can still depend on operating-system and library details for tree-based learners. The reproducibility target is therefore the data-cleaning rules, reconstructed cohort, split definitions, feature construction, and reported metrics within ordinary numerical tolerance. The recorded environment in Table 17 is provided to make that target as concrete as possible.

IX. CONCLUSION

Reconstructed Jira history contains measurable signal for two sprint-start outcomes, but the signal is narrower than a generic “sprint risk” claim. Late closure is only moderately discriminable and varies across projects. For nonempty sprints, initial scope is strongly associated with unfinished work; a simple scope-only logistic model reaches 0.885 AUC and does not lose to the more complex random forest on the main split. Stricter provenance requirements reduce the sample and weaken some estimates, which makes data history part of the result rather than a footnote. The practical value of these rankings remains a question for prospective validation with real teams.

TABLE XVII: Recorded software environment

Component	Version
Python	3.13.5
NumPy	2.3.5
pandas	2.2.3
scikit-learn	1.8.0
XGBoost	3.1.3

REFERENCES

- [1] V. Tawosi, A. Al-Subaihin, R. Moussa, and F. Sarro, “A versatile dataset of agile open source software projects,” in *Proc. 19th Int. Conf. Mining Software Repositories*, 2022, pp. 707–711.
- [2] M. Choetkiertikul, H. K. Dam, T. Tran, T. T. M. Pham, A. Ghose, and T. Menzies, “A deep learning model for estimating story points,” *IEEE Trans. Softw. Eng.*, vol. 45, no. 7, pp. 637–656, 2019.
- [3] S. P. Shankar, S. S. Chaudhari, V. Mishra, and T. Stephan, “Intelligent techniques for predictive analytics in Agile software development,” *Scientific Reports*, vol. 16, 2026.
- [4] Y. J. Pérez Castillo, S. D. Orantes Jiménez, and P. O. Letelier Torres, “Sprint management in Agile approach: Progress and velocity evaluation applying machine learning,” *Information*, vol. 15, no. 11, 2024.
- [5] P. G. Obike, V. E. Ekong, and O. U. Obot, “A novel scoring-based Agile sprint deliverability prediction and prioritization using machine learning,” *Int. J. Comput. Appl.*, vol. 187, no. 49, 2025.
- [6] S. S. Almalki, “AI-driven decision support systems in Agile software project management,” *Systems*, vol. 13, no. 3, 2025.
- [7] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. ACM SIGKDD*, 2016, pp. 785–794.
- [8] L. Breiman, “Random forests,” *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.
- [9] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.